

KARADENIZ TECHNICAL UNIVERSITY — GRADUATE SCHOOL OF NATURAL AND APPLIED SCIENCES

Constraint Programming

Modeling Languages II: The Mathematical Programming View

Week 7

Modeling Languages II: The Mathematical Programming View

Assoc. Prof. Tolga Berber

KTU Faculty of Science, Department of Computer Science · https://avesis.ktu.edu.tr/tberber

Learning phases: 1 • Pre-instruction 2 • Content 3 • Participation 4 • Assessment 5 • Follow-through

1 • Pre-instruction Why It Matters?

Every model you have built so far answers “is there a solution, or here are all of them.”

- Week 8 will eventually add `Minimize`/`Maximize` to CP-SAT – but mathematical programming has been objective-first from the start
- Imagine a factory manager who does not want *a* production plan that respects the labour and material limits
- The manager wants provably the most profitable plan, and proof that no better plan exists

This unit gives you the language that question is written in.

This Week

◎ Learning Objectives

By the end of this week, you will be able to:

1. Define an LP and an IP/MIP; explain what an LP relaxation is and why it bounds the MIP's optimal value
2. Explain how branch-and-bound uses a relaxation's bound to prune, and why that proves optimality
3. Build and run first `gurobipy` models: assignment, knapsack and production planning
4. Compare a CP-SAT and a `gurobipy` formulation of the same problem and choose between the two paradigms
5. Translate a paper LP/MIP formulation into a verified, running `gurobipy` program

SECTION 1

Unit 1 — Linear and Integer Programs, and LP Relaxation

2 • Content From CSP to LP: What Changes

- Week 1's formal triple: variables with arbitrary finite domains, constraints, no built-in notion of "better"
- An LP's decision variables are continuous by default, and an objective is not optional – optimization is the starting point

Definition: Linear program (LP)

Continuous decision variables, a linear objective function (minimize/maximize a weighted sum), and linear constraints ($\leq, \geq, =$ on weighted sums), plus non-negativity.

Analogy: A Budget Spreadsheet, Two Kinds of Lines

Analogy

A travel budget might let you allocate any fractional amount to “meals” – a continuous line item – but a “conference registration” line is paid in full or not at all. Mixing the two kinds of line items in one spreadsheet is exactly what a mixed-integer program mixes in one model.

An **IP** restricts every variable to integers; a **MIP** mixes continuous and integer (or binary) variables in one model.

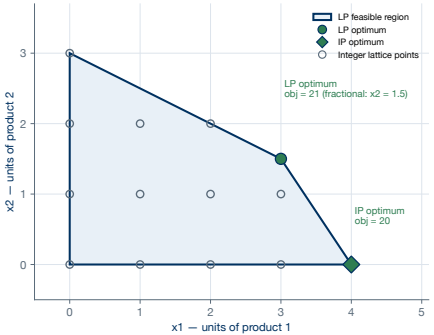
The Feasible Region and LP Relaxation

- **Feasible region:** the set of points satisfying every constraint – for an LP, a polytope whose optimum sits at one of its corners (vertices)

Definition: LP relaxation

Given a MIP, drop every integrality restriction (integer/binary $\rightarrow$ continuous), keeping the same objective and constraints. Its optimal value is always a valid bound on the MIP's optimal value – because every integer-feasible point is also LP-feasible.

A Bound, Not Necessarily an Achievable Answer



A two-variable toy instance: the LP optimum sits at a fractional vertex; the true integer optimum sits at a different lattice point entirely.

Worked by Hand: A 0/1 Knapsack

Capacity 6; three items, each with a weight and a value.

- **A:** weight 2, value 3 **B:** weight 3, value 4 **C:** weight 4, value 5
- **IP:** maximize $3x_A + 4x_B + 5x_C$ subject to $2x_A + 3x_B + 4x_C \leq 6$, $x_A, x_B, x_C \in \{0, 1\}$
- Enumerating all 8 subsets by hand: the true integer optimum is $\{A, C\}$, value 8, weight 6

Worked by Hand: The LP Relaxation

Same objective and constraint; only the domain changes to $0 \le x_A, x_B, x_C \le 1$.

- Value-per-weight ratios: A $= 1.5$, B ≈ 1.333, C $= 1.25$ – fill greedily in that order
- $x_A = 1$ (weight 2, 4 left), $x_B = 1$ (weight 3, 1 left), $x_C = 0.25$ (weight 1, 0 left)
- Relaxed optimum 8.25 – strictly above the true integer optimum of 8

A relaxation's bound is not itself an achievable integer answer.

3 • Participation Quick Check: State and Solve the Relaxation

Using the knapsack instance above, state exactly which part of the IP formulation changes in its LP relaxation, then compute the relaxation's optimum by hand.

Only the domain changes: $x_A, x_B, x_C \in \{0, 1\}$ becomes $0 \leq x_A, x_B, x_C \leq 1$; the objective and constraint stay identical. Greedy fill by ratio gives value $3 + 4 + 1.25 = 8.25$ – the fractional variable is x_C.

SECTION 2

Unit 2 — Branch-and-Bound and Why MIP Works

2 • Content The Branch-and-Bound Algorithm

- Solve the LP relaxation at the current node
- Integer-feasible? It is a candidate solution – an incumbent if it beats the best one found so far
- Fractional? Pick a variable and **branch**: e.g. for binary x_C, one child fixes $x_C \leq 0$, the other $x_C \geq 1$
- Recurse on each child

Bounding and Pruning

Analogy

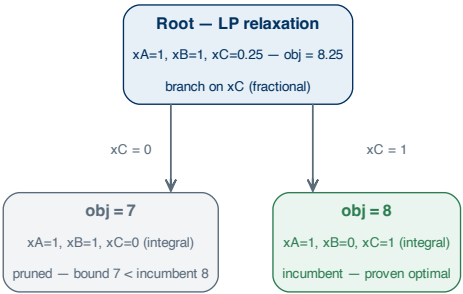
Once you have a fully booked itinerary at a known total cost, and a half-planned alternative whose cheapest-possible completion is already more expensive, you stop planning that alternative further – you already know it cannot win.

A child's relaxation value bounds everything reachable in its subtree. The moment a bound is no better than the incumbent, that entire subtree is pruned without being explored further.

Two Ways to Prune a Search Tree

- **CP-SAT (Weeks 2–4):** prunes using constraint propagation over discrete domains – arc consistency, global-constraint reasoning
- **Branch-and-bound:** prunes using a continuous LP relaxation's numeric bound
- Two different sources of the same idea: "don't explore a branch that provably cannot help"

Full Trace: The Knapsack Instance



Root relaxation 8.25, fractional on x_C – branch. $x_C{=}0$ gives an integral 7, pruned once $x_C{=}1$'s integral 8 becomes the incumbent. Proven optimum: 8.

3 • Participation Quick Check: Does Pruning Still Hold?

The $x_C = 0$ child's relaxation happened to be integer-feasible at value 7. Would pruning it against an incumbent of 8 still have been correct if that bound of 7 had *not* been integral? Why?

Yes – pruning only depends on the bound being no better than the incumbent, never on whether the bound is itself already integral. A fractional bound of 7 is still a valid upper bound on the subtree, and 7 cannot beat an incumbent of 8 regardless of how remaining fractional variables would eventually resolve.

SECTION 3

Unit 3 — First Models in gurobipy

2 • Content gurobipy's Building Blocks

Deliberately parallel to Week 1's CP-SAT blocks.

- `gp.Model()` – compare `cp_model.CpModel()`
- `model.addVar(vtype=..., name=...)` – compare `NewIntVar(...)`
- `model.setObjective(expr, GRB.MAXIMIZE)` – new: CP-SAT has no direct analogue yet
- `model.addConstr(...)` – compare `model.Add(...)`
- `model.optimize()`, then `model.ObjVal` and a variable's `.X` – compare `solver.Value(var)`

Three vtypes You Need This Unit

- `GRB.CONTINUOUS` – the LP-relaxation style variable from Unit 1
- `GRB.INTEGER` – whole-number quantities (production planning)
- `GRB.BINARY` – the $\{0, 1\}$ case knapsack and assignment both need

Reading status: `model.Status == GRB.OPTIMAL` – the same status-checking habit from Week 1 transfers directly.

Code: Knapsack as a Binary IP

```python
model = gp.Model("knapsack_ip")
x = {item: model.addVar(vtype=GRB.BINARY, name=item) for item in
    ITEMS}
```

```python
model.addConstr(
   gp.quicksum(WEIGHT[item] * x[item] for item in ITEMS) <= CAPACITY,
   name="capacity",
)
```

Unit 1's IP formulation, written directly into code: three `GRB.BINARY` variables, one capacity constraint.

Code and Output: Solving the IP

```python
model.optimize()
status = "OPTIMAL" if model.Status == GRB.OPTIMAL else str(model.Status)
print(f"Status: {status}")
print(f"Optimal value: {model.ObjVal}")
for item in ITEMS:
    print(f" {item}: {x[item].X}")
```

Expected Output

```
Status: OPTIMAL
Optimal value: 8.0
  A: 1.0
  B: 0.0
  C: 1.0
```

Items A and C – exactly Unit 1's hand-enumerated optimum.

Code: The LP Relaxation

```
model = gp.Model("knapsack_lp_relaxation")
x = {
   item: model.addVar(vtype=GRB.CONTINUOUS, lb=0, ub=1, name=item)
   for item in ITEMS
}
add_capacity_constraint(model, x)
model.setObjective(
   gp.quicksum(VALUE[item] * x[item] for item in ITEMS), GRB.MAXIMIZE
)
model.optimize()
print(f"Relaxed optimal value: {model.ObjVal}")
for item in ITEMS:
   print(f" {item}: {x[item].X}")
```

Same three items, capacity and objective – only the `vtype` and bounds change from Unit 1's binary formulation.

Output: The LP Relaxation, For Real

Expected Output

```
Relaxed optimal value: 8.25
  A: 1.0
  B: 1.0
  C: 0.25
```

Same fractional variable, same value as your by-hand computation.

Code: Assignment – Variables and Constraints

```
model = gp.Model("assignment")
x = model.addVars(WORKERS, TASKS, vtype=GRB.BINARY, name="x")
```

```
for worker in WORKERS:
    model.addConstr(gp.quicksum(x[worker, task] for task in TASKS) ==
        1)
for task in TASKS:
    model.addConstr(gp.quicksum(x[worker, task] for worker in WORKERS)
         == 1)
```

Two constraint families: every worker does exactly one task, every task gets exactly one worker.

Code: Solving the Assignment

```
model.optimize()
assignment = {}
for worker in WORKERS:
   for task in TASKS:
      if x[worker, task].X > 0.5:
         assignment[worker] = task
         print(f"{worker} -> {task}")
print(f"Total cost: {model.ObjVal}")
```

Reading each binary `x[worker, task]` back with `.X > 0.5` recovers the chosen pairing.

Output: Solving the Assignment

Expected Output

```
W1 -> Ta
W2 -> Tb
W3 -> Tc
W4 -> Td
Total cost: 14.0
```

The cost matrix's diagonal – unambiguous by inspection, confirmed by a real solver run.

What You Will See: The Restricted-Licence Banner

No `gurobi.lic` present → `gurobipy` falls back to pip's built-in size-limited licence.

Expected Output

```
Restricted license - for non-production use only - expires 2026-11-23
Gurobi Optimizer version 12.0.3 build v12.0.3rc0
```

- Expected, real output – not an error to hide
- Every model this week stays far under the 2000-variable/ constraint cap

Code and Output: Production Planning

```python
model.optimize()
plan = {product: x[product].X for product in PRODUCTS}
for product in PRODUCTS:
    print(f"{product}: {plan[product]}")
print(f"Total profit: {model.ObjVal}")
```

Expected Output

```
P1: 9.0
P2: 0.0
P3: 13.0
Total profit: 620.0
```

Three `GRB.INTEGER` variables, two resource-budget constraints – you cannot ship half a manufactured unit.

3 • Participation Quick Check: Fix the Missing Constraints

A partially written assignment script only has “every worker does exactly one task” – the “every task gets exactly one worker” family is missing. Write the missing pattern.

`for j in tasks: model.addConstr(gp.quicksum(x[i, j] for i in workers) == 1)` – with both families present, the script reports the unique optimal assignment at total cost 14.

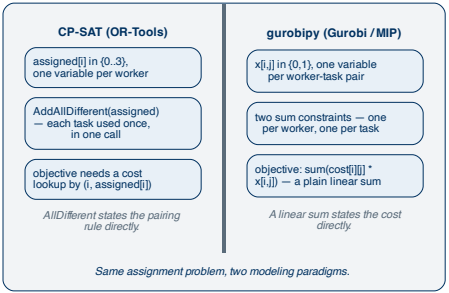
SECTION 4

Unit 4 — CP vs MIP: The Same Problem, Two Ways

2 • Content The Same Assignment Problem, Two Ways

- **CP-SAT sketch:** one variable `assigned[i]` per worker, domain $\{0..3\}$; a single `model.AddAllDifferent(assigned)` directly captures “each task used once”
- Its cost objective needs an `AddElement`-style lookup – cost depends on both i and the *value* of `assigned[i]`
- **gurobipy (Unit 3):** two families of linear equality constraints state the same idea; the cost objective is a plain linear sum, no lookup needed

Same Problem, Two Panels



`AllDifferent` states the pairing rule directly; a linear sum states the cost directly – each paradigm wins on a different half of the same problem.

A Rule of Thumb, Not a Table to Memorize

Analogy

Choosing between CP-SAT and `gurobipy` is like choosing between two travel-planning tools: one enforces complicated combination rules directly ("no two connecting flights may overlap"); the other optimizes a cost number and tells you exactly how much cheaper a different budget would make your trip.

Both engines share one property Week 4's local search does not: when they say "optimal," it is a proof, not a best-effort claim.

3 • Participation Quick Check: Sketch and Compare

Sketch, in words, a CP-SAT model of the assignment problem: which variables, which global constraint, what stands in for the objective? Which model more directly expresses the pairing rule – which expresses the cost?

`AllDifferent` more directly expresses “each task used once” in one call; `gurobipy`’s `sum(cost[i][j] * x[i, j] ...)` more directly expresses the cost objective than CP-SAT’s lookup-based construction would.

SECTION 5

Summary

Summary

Summary

- An LP relaxation drops integrality but keeps objective and constraints – its optimum is always a valid bound on a MIP's optimal value
- Branch-and-bound prunes a subtree the moment its relaxation's bound cannot beat the current incumbent – a proof, not a guess
- `gurobipy` builds continuous, integer and binary models with the same recipe as CP-SAT: model, variables, constraints, objective, solve
- CP-SAT and `gurobipy` each express a different half of the same problem more directly – neither is universally better

4 • Assessment Check Yourself

- One Check Yourself question closes each unit in this week's chapter – answers are collected at the chapter's end
- A second knapsack instance's IP and LP relaxation, pruning without an integral bound, a two-product production plan in words, a CP-SAT vs. `gurobipy` rule of thumb

5 • Follow-through Transfer — Where Else?

- Week 10 attaches a full sensitivity report to these exact three problems – knapsack, assignment, production planning – turning `Pi` and `RC`, only named this week, into an entire chapter
- Week 12 reuses `gurobipy`'s modelling patterns for rostering, and combines CP-SAT with MIP in logic-based Benders decomposition
- Week 14 reuses the same patterns for facility location
- This course has no running capstone project – the thread is the *same* classic problems, revisited with new machinery

In the Lab

This week's lab sheet

You build and run all three `gurobipy` models yourself, hand-trace a full branch-and-bound search, catch and fix a genuinely broken assignment script by running it, and close by building a travel-budget MIP completely from scratch.

- Next week: CP-SAT finally gets a formal objective – `Minimize`/`Maximize` for over-constrained and weighted problems
- Week 10: this week's LP relaxation grows into duality, shadow prices and a full what-if analysis